SHARE WITH THE WHOLE TEAM

Automated Municipal Ticket Team Task Guide

Five clear assignments: what to research, what to build, and what working prototype to share.

Main goal: A citizen sends a photo and a short report about a city problem. Our system understands it, finds the correct municipal procedure, chooses the right department, and creates a ticket that municipal staff can review.

How the system works



n8n connects these parts. Policy Gates check for missing data, unclear results, and weak sources. Rules, not the AI model, choose the department.

Who owns what

01	**n8n & system integration** Build the full workflow and connect every module.
02	**RAG & Knowledge Agent** Find municipal documents and retrieve trusted procedures.
03	**Intake & privacy** Process citizen media and clean personal information.
04	**Vision & prompts** Understand images and test/improve AI prompts.
05	**Web app & database** Build the forms, dashboard, and saved tickets.

Important: Start with a small working version. Research only what you need to make a good choice, then build and test. Use mock outputs if another module is not ready. Do not wait for the whole team to finish.

Project focus: n8n automation, prompts with before/after tests, your own RAG knowledge base, three cooperating agents, and real image processing. Keep advanced extras for later.

MESSAGE TO MEMBER 1

n8n, workflow & system integration

Your job: Build the central workflow so all parts of the project work together. You coordinate how data moves; you do not need to build the other members' agents.

Research - learn and decide

- **n8n tools:** Learn Webhook, HTTP Request, Code, IF/Switch, Postgres, error handling, and how to export a workflow.
- **Agent design:** Compare a simple sequential chain with a central master agent. Explain why a sequence fits our project.
- **Safe data flow:** Learn JSON validation, timeouts, retries, logging, and what to do when an agent fails or returns missing fields.
- **Routing:** Plan simple rules that map road damage, street lighting, waste, and water leaks to the correct department.

Build - your prototype

- **Agree on JSON fields** with the team and create **docs/contracts.md**. Keep field names the same across all modules.
- **Create an n8n workflow:** webhook → input check → Intake → Gate 1 → Vision → Gate 2 → department rules → Knowledge/RAG → Gate 3 → ticket creation → response.
- **Use sample (mock) agent results** until real endpoints are ready. All branches should still run from one webhook.
- **Make PASS / REVIEW / REJECT decisions:** pass valid data, send unclear cases for a human check, stop truly invalid reports.
- **Log key events:** report ID, result, errors, and agent/prompt versions. Keep API keys out of shared files.

Files and working deliverable

- **n8n/municipal-ticket.json** - workflow anyone can import.
- **docs/contracts.md** - request/response examples, field types, and allowed values.
- **docs/research/architecture.md** - short explanation of tools and design choices.
- **README + sample curl request** - how to launch the workflow and test it.

Show it working

Prototype test: Send a sample report to the webhook. Show a ticket-ready result for a valid report, REVIEW for a doubtful report, and REJECT for missing required input.

Work with: Members 2, 3, and 4 for agent endpoints; Member 5 for saving the ticket and showing it in the app.

MESSAGE TO MEMBER 2

RAG, documents & Knowledge Agent

Your job: Build the part that finds a useful municipal procedure for an incident and shows where that procedure came from.

Research - learn and decide

- **Document sources:** Find trustworthy municipal or public-service documents for roads, lighting, waste, and water leaks. Record their original links.
- **Choose a municipality:** Define the location our documents apply to. Never present made-up rules or repair deadlines as official.
- **RAG basics:** Compare splitting documents by headings/procedures with splitting by fixed word/token size.
- **Search setup:** Compare multilingual embedding options; learn Qdrant, top-k results, and filters by category/department.
- **Evaluation:** Learn how to check whether the expected document appears first or among the top three search results.

Build - your prototype

- **Collect a small corpus:** Aim for around 8-15 useful document sections, if available. Save source title, URL, category, department, and municipality.
- **Make a document pipeline:** documents → chunks → embeddings → Qdrant. Start with structure-based chunks around 400-500 tokens and 50 overlap.
- **Add a comparison method:** Also test fixed chunks around 512 tokens with 50 overlap on the same documents and queries.
- **Create the Knowledge Agent:** Given an incident category, department, and description, search top 3 results and write a short procedure based only on them.
- **Return clear proof:** Include procedure, source document, chunk ID, search score, and deadline only when the document supports one. Otherwise set it to null.
- **Test retrieval:** Start with 5 labeled questions and a no-answer case; expand the evaluation with more queries after the core module works.

Files and working deliverable

- **rag/corpus/ + sources.csv** - documents and proof of where they came from.
- **rag/ingest.py** - runs chunking, embeddings, and Qdrant indexing.
- **POST /knowledge** - callable endpoint returning JSON with sources.
- **evaluation/rag_results.csv** - questions, expected sources, retrieved results.
- **prompts/knowledge/** and README - prompt versions and run instructions.

Show it working

Prototype test: Ask about a pothole. Show the retrieved municipal procedure, source document, and chunk ID. Ask a question with no matching source and show that the agent does not invent an answer.

Work with: Member 1 for JSON and n8n; Member 4 to record and improve the Knowledge Agent prompts.

MESSAGE TO MEMBER 3

Intake Agent, image input & privacy

Your job: Build the part that receives a citizen report and turns it into clean, structured input for the Vision Agent.

Research - learn and decide

- **Media handling:** Learn how to accept image files, check type and size, resize them, and reject broken uploads.
- **Audio tools:** Compare Whisper or another speech-to-text option for a later image + audio path.
- **Video tools:** Learn basic FFmpeg frame/audio extraction, but do not make video necessary for the first working version.
- **Privacy:** Learn basic face blurring (for example OpenCV), text masking for emails/phone numbers, and safe media storage.
- **Agent scope:** Separate simple file checks from AI work such as language understanding or cleaning an unclear description.

Build - your prototype

- **Start with image + text:** Receive the media, description, report ID, and optional location. Check required fields and file format.
- **Normalize the report:** Return consistent JSON with report ID, input type, cleaned text, language, media reference, location, validity, and errors.
- **Make a privacy-safe copy:** Blur faces when detected and hide obvious personal details in text. Keep analysis data separate from stored/displayed data.
- **Handle mistakes:** Return clear errors for missing images, unsupported formats, unreadable files, and empty required descriptions.
- **Prepare for integration:** Give Member 4 a reliable way to read the validated image. Share media references with Member 5.
- **Extra after core works:** Add audio transcription; later, video frames if time and reliability allow.

Files and working deliverable

- **POST /intake** - endpoint or callable Python module.
- **agents/intake/** - image checks, preprocessing, normalization, and privacy code.
- **tests/intake/** - at least 5 examples, including bad inputs.
- **docs/research/intake.md** - tools selected and their limits.
- **prompts/intake/** (if an LLM is used) and a README.

Show it working

Prototype test: Submit a photo and description and show clean JSON. Submit an invalid file and show a clear error. Show an example where a detected face is blurred in the stored copy.

Work with: Member 1 for the JSON format, Member 4 for Vision input, Member 5 for uploads and safe display of images.

MESSAGE TO MEMBER 4

Vision Agent & prompt experiments

Your job: Build the AI that reads the photo, identifies the city problem, and gives evidence for its choice. Help the team document prompt improvements.

Research - learn and decide

- **Vision models:** Compare at least 2 image-understanding model options for quality, cost, speed, and JSON support.
- **Prompts:** Learn zero-shot/few-shot prompts, strict JSON outputs, fixed category labels, and how to reduce invented details.
- **Failure cases:** Think about unrelated images, blurry photos, and descriptions that do not match the photo.
- **Evaluation:** Learn category accuracy, correct rejection, valid JSON rate, and how to compare prompts fairly on the same images.

Build - your prototype

- **Agree on labels:** ROAD_DAMAGE, STREET_LIGHTING, WASTE, WATER_LEAK, NONE. Start with LOW, MEDIUM, and HIGH severity.
- **Build POST /vision:** Input is validated image + description + report ID. Output includes category, severity, short description, evidence, uncertainty/confidence, and text-image match.
- **Try prompt versions:** v1 simple classification; v2 strict JSON and allowed labels; v3 adds NONE, visible evidence, and mismatch checks.
- **Test fairly:** Gather around 10 labeled photos for the prototype; expand toward 20-30 with valid, invalid, unclear, and mismatch cases.
- **Keep results:** Record expected label, model result, prompt version, errors, and what changed. Do not assume a later prompt is better until tests show it.
- **Help other agents:** Give Members 2 and 3 a simple prompt experiment template, then combine the team's final prompt library.

Files and working deliverable

- **POST /vision** - callable image classification endpoint.
- **prompts/vision/v1.md, v2.md, v3.md** - prompt versions with notes.
- **evaluation/vision_results.csv** - labeled test cases and results.
- **docs/research/vision_prompts.md** - model choices, failures, and improvements.
- **prompts/README.md** - shared template for all agent prompts.

Show it working

Prototype test: Show a pothole image classified as road damage with visible evidence. Show an unrelated photo marked NONE or uncertain. Show an image/text mismatch and compare two prompt versions.

Work with: Member 3 for input images, Member 1 for policy checks, Member 2 for category context and RAG prompt records.

MESSAGE TO MEMBER 5

Citizen web app, dashboard & database

Your job: Build the screens people use and save the tickets that come out of the AI workflow. Keep the first version simple and functional.

Research - learn and decide

- **Web app:** Choose a fast, familiar framework (for example React + Vite). Learn file upload and simple form checks.
- **Location:** Learn browser GPS permission and a manual location option when GPS is not available.
- **Database:** Learn PostgreSQL tables, ticket states, and how to store file paths instead of large image files in a table.
- **API calls:** Learn how the web app calls n8n and how to show loading, success, review, and error results.
- **Testing:** Learn simple API tests and checks for ticket changes.

Build - your prototype

- **Citizen page:** Image upload, text description, optional location, Submit button, and a ticket result/tracking ID.
- **Municipality page:** List tickets and show category, severity, priority, department, status, anonymized image, and RAG source.
- **Review actions:** Make basic Approve, Reject, Assign, and Resolve controls that update stored data.
- **Database:** Make reports, tickets, and preferably ticket_events tables. Save details from the workflow and each important status change.
- **Connect the API:** Use a mock n8n reply at the start. Then switch to Member 1's real webhook without changing the screen design.
- **Keep media safe:** Show anonymized images only on the municipality dashboard, not raw uploaded copies.

Files and working deliverable

- **frontend/** - working citizen form and municipality dashboard.
- **database/schema.sql** or migrations - tables and test data.
- **API code** - submit a report, save a ticket, read tickets, and change status.
- **tests/app/** - at least a successful submission and a review case.
- **README** - setup, environment variables, and how to open the app.

Show it working

Prototype test: Submit an image and description in the citizen page. Receive a ticket ID from the mock workflow. Show the saved ticket in the dashboard, then change its status and refresh to prove it was saved.

Work with: Member 1 for webhook and ticket format, Member 3 for media references, Member 2 to display procedure sources.

SHARED AGREEMENT

How our components connect

> **Rule for everyone:** Use the same field names and allowed values. Member 1 owns the shared contract document, but all members must agree to it before changing an endpoint.

The minimum API connections

Citizen app → n8n	Report ID, photo reference/upload, description, optional location
n8n → Intake Agent	Raw report details and media reference
Intake → Vision Agent	Validated image reference and cleaned description
Vision → n8n router	Category, severity, evidence, match result, uncertainty
n8n → Knowledge Agent	Validated category, department, and incident description
Knowledge → ticket storage	Procedure and source metadata; deadline only if documented
n8n / database → dashboard	Ticket ID, status, routing and anonymized evidence

Agree on these values

- **Incident categories:** ROAD_DAMAGE, STREET_LIGHTING, WASTE, WATER_LEAK, NONE.
- **Severity:** LOW, MEDIUM, HIGH; unknown values should be handled safely.
- **Validation outcome:** PASS, REVIEW, or REJECT. REVIEW means a person must check it; REJECT means the request is not usable.
- **Ticket status:** RECEIVED, VALIDATING, ANALYZING, REVIEW_REQUIRED, TICKET_CREATED, ASSIGNED, IN_PROGRESS, RESOLVED (plus REJECTED if needed).
- **Missing information:** Use null or an explicit unknown value. Never invent locations, official procedures, or deadlines.

Example: Vision Agent output

```
{
  "report_id": "RPT-001",
  "category": "ROAD_DAMAGE",
  "severity": "HIGH",
  "description": "Pothole visible on the road",
  "evidence": ["Hole visible in the road surface"],
  "text_image_consistent": true,
  "confidence": 0.91,
  "prompt_version": "v2"
}
```

The numbers above are only an example. Model confidence must not be treated as a guarantee of correctness.

FOR ALL FIVE MEMBERS

What counts as finished?

Every member must share a small, real prototype. A document with only ideas is not enough. Your code should run on another team member's computer using the README.

Each member must hand over

- [ ] **Research note:** What you looked at, what you chose, and why.
- [ ] **Working code:** A script, endpoint, workflow, screen, or database that really runs.
- [ ] **How to run it:** Setup steps, environment variable names, and any safe example data.
- [ ] **Test proof:** At least one case that works and one that fails or needs review.
- [ ] **Example output:** Sample JSON, screenshot, test results, or a short demo.
- [ ] **Handoff:** Tell the next member how to call your module and what it returns.

Shared demo scenarios

1. Normal report Citizen sends a pothole photo and matching text. System routes it, finds a procedure with a source, and creates a ticket.

2. Image/text mismatch Citizen says pothole but sends a broken streetlight photo. System sends the case for human REVIEW.

3. Unclear or invalid report Image is missing, broken, or uncertain. System asks for correction, REJECTS invalid input, or sends uncertainty to REVIEW.

Work faster without breaking integration

- **Build in parallel.** If another API is missing, use mock JSON with the agreed contract.
- **Keep changes small.** Share runnable code often; test the connection between modules as soon as both sides exist.
- **Save experiments.** Keep prompt versions, RAG search results, bad examples, and what you changed to fix them.
- **Focus on the core.** Image + text, 3 agents, RAG, n8n, real tickets, and validation come before advanced video, Telegram, maps, or extra features.
- **Everyone learns the full system.** Each person should be able to explain their own work and how the other parts connect.

Shared target: A citizen can submit a real image and report, the three agents can process it through n8n, and a safe, traceable ticket appears in the municipality dashboard.

Based on the team's Automated Municipal Ticket solution document and the Advanced Deep Learning project requirements.